



datasetManager

Application console de gestion de jeux de données — Rapport de réalisation

Module p1ia — Application Python

Dépôt GitHub : github.com/sirius-code7/datasetManager

Auteur : papa

Sommaire

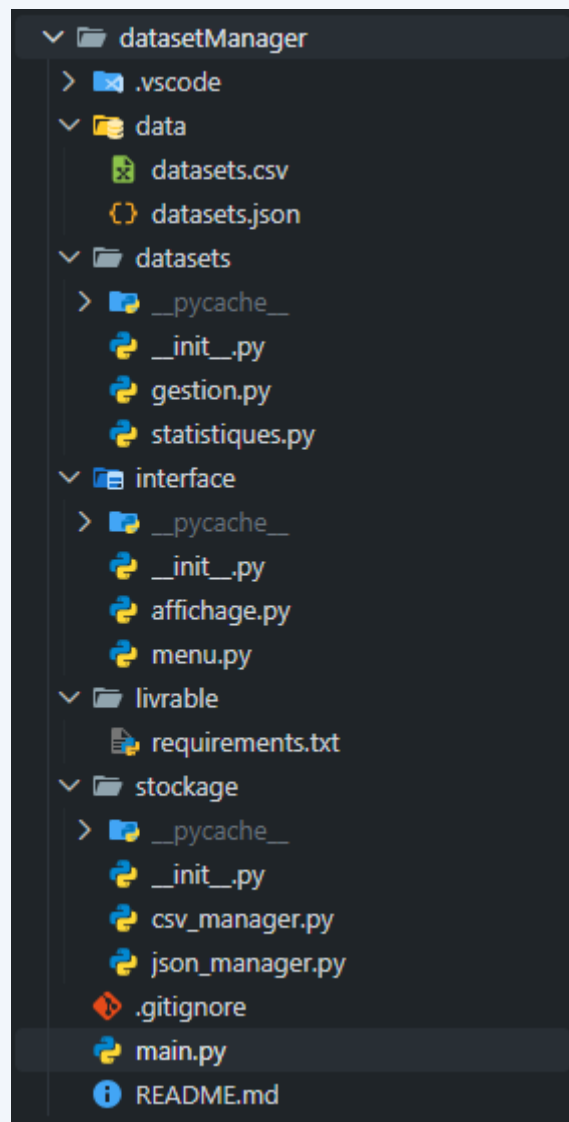
Sommaire	2
Contexte du projet	3
Aperçu du dépôt	3
Partie 1 — Types de base, variables, entrées et sorties	4
Partie 2 — Structures de contrôle	4
Partie 3 — Dictionnaires	5
Partie 4 — Tuples	6
Partie 5 — Listes	7
5.1 — Ajouter	7
5.2 — Trier	8
5.3 — Rechercher	9
5.4 — Modifier	10
5.5 — Supprimer	10
Partie 6 — Compréhensions (listes et dictionnaires)	11
Partie 7 — Fichiers	12
7.1 — Sauvegarde	12
7.2 — Rechargement et affichage	13
Partie 8 — Exceptions	14
8.1 — Validation des saisies (nombres, texte)	14
8.2 — Fichier introuvable	16
8.3 — Dataset introuvable	17
8.4 — Fichier vide	18
Partie 9 — Fonctions	19
Partie 10 — Modules	21
Partie 11 — Packages	21
Partie 12 — Bonus	22
Conclusion	24

Contexte du projet

Ce projet consiste à développer progressivement, en tant que nouvel arrivant dans une entreprise spécialisée en Intelligence Artificielle, une application console Python permettant de gérer un catalogue de jeux de données (CSV/JSON) : enregistrement des caractéristiques, recherche, statistiques, sauvegarde et rechargement des fichiers, gestion des erreurs. L'application a été construite étape par étape, avec un commit GitHub clair pour chaque partie du cahier des charges.

Aperçu du dépôt

Structure finale du dépôt tel que poussé sur GitHub :



Vue d'ensemble de l'arborescence du dépôt (VS Code / explorateur de fichiers).

Commit(s) associé(s) : organiser l'application en packages (partie 11.)

Partie 1 — Types de base, variables, entrées et sorties

Saisie des métadonnées d'un dataset (nom, domaine, nombre de lignes, nombre de colonnes, taille en Mo, format, caractère public ou non) puis affichage d'un résumé formaté à l'écran.

```
C:\Users\Invite\Downloads\python\datasetManager>python main.py
Nom du dataset : ia
Domaine (Santé, Finance, Agriculture, Transport, Education ...) : tech
Nombre de lignes : 12
Nombre de colonnes : 24
Taille en Mo : 10.5
Format (csv ou json) : csv
Public ? (true/false) : true

===== Résumé du dataset =====
Nom          : ia
Domaine      : tech
Lignes       : 12
Colonnes     : 24
Taille       : 10.5 Mo
Format       : CSV
Public       : Oui
=====
```

Saisie des métadonnées et résumé formaté affiché dans la console.

Commit(s) associé(s) : utiliser la librairie cs50 pour les saisies utilisateur avec interpréteur python (partie 1.)

Partie 2 — Structures de contrôle

Mise en place d'un menu interactif provisoire (Ajouter, Afficher, Rechercher, Quitter) qui garde le programme actif tant que l'utilisateur ne choisit pas de quitter.

```

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher
4. Quitter
=====
Votre choix : 1
→ (à venir) Ajouter un dataset

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher
4. Quitter
=====
Votre choix : 4
Au revoir !

C:\Users\Invite\Downloads\python\datasetManager>|

```

Menu interactif affiché dans la console.

Commit(s) associé(s) : ajout du menu interactif avec boucle et structures de contrôle (partie 2.)

Partie 3 — Dictionnaires

Les métadonnées de chaque dataset sont désormais stockées dans un dictionnaire (nom, domaine, lignes, colonnes, taille, format, public), en remplacement des variables séparées de la Partie 1 — sans changer le comportement visible côté console.

```
# --- Partie 3 : Stockage dans un dictionnaire ---
dataset = {
    "nom": nom,
    "domaine": domaine,
    "lignes": lignes,
    "colonnes": colonnes,
    "taille": taille,
    "format": format_fichier.upper(),
    "public": public
}

# --- Affichage du résumé formaté (à partir du dictionnaire) ---
print("\n==== Résumé du dataset =====")
print(f"Nom      : {dataset['nom']}")
print(f"Domaine   : {dataset['domaine']}")
print(f"Lignes    : {dataset['lignes']}")
print(f"Colonnes  : {dataset['colonnes']}")
print(f"Taille    : {dataset['taille']} Mo")
print(f"Format    : {dataset['format']}")
print(f"Public    : {'Oui' if dataset['public'] else 'Non'}")
print("=====\\n")
```

Refactorisation vers un dictionnaire : le comportement console reste identique.

Commit(s) associé(s) : `stockage des métadonnées dans un dictionnaire (partie 3.)`

Partie 4 — Tuples

Création d'un tuple des domaines autorisés (Santé, Finance, Agriculture, Transport, Education) et vérification que le domaine saisi par l'utilisateur en fait bien partie.

```

C:\Users\Invite\Downloads\python\datasetManager>python main.py
Nom du dataset : ia
Domaine ('Santé', 'Finance', 'Agriculture', 'Transport', 'Education') : education
Domaine invalide, choisissez parmi la liste proposée.

Domaine ('Santé', 'Finance', 'Agriculture', 'Transport', 'Education') : Education
Nombre de lignes : 7
Nombre de colonnes : 5
Taille en Mo : 5.10
Format (csv ou json) : csv
Public ? (true/false) : true

==== Résumé du dataset ====
Nom      : ia
Domaine  : Education
Lignes   : 7
Colonnes : 5
Taille   : 5.1 Mo
Format   : CSV
Public   : Oui
=====

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher
4. Quitter
=====
Votre choix : 4
Au revoir !

C:\Users\Invite\Downloads\python\datasetManager>

```

Vérification du domaine saisi par rapport au tuple des domaines autorisés.

Commit(s) associé(s) : tuples contenant les domaines autorisés (partie 4

Partie 5 — Listes

Les datasets sont désormais stockés dans une liste de dictionnaires. Chaque ajout est enregistré dans la liste, avec les fonctionnalités : ajouter, trier, rechercher, modifier, supprimer.

5.1 — Ajouter

```

C:\Users\Invite\Downloads\python\datasetManager>python main.py
=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher
4. Quitter
=====
Votre choix : 1
Nom du dataset : Neurologie
Domaine ('Santé', 'Finance', 'Agriculture', 'Transport', 'Education') : Santé
Nombre de lignes : 33
Nombre de colonnes : 19
Taille en Mo : 100.2
Format (csv ou json) : csv
Public ? (true/false) : false

==== Résumé du dataset ====
Nom       : Neurologie
Domaine   : Santé
Lignes    : 33
Colonnes  : 19
Taille    : 100.2 Mo
Format    : CSV
Public    : Non
=====
Dataset 'Neurologie' ajouté avec succès.

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher
4. Quitter
=====

```

Ajout d'un nouveau dataset à la liste.

5.2 — Trier

```

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Trier les datasets (à venir)
4. Rechercher un dataset (à venir)
5. Modifier un dataset (à venir)
6. Supprimer un dataset (à venir)
7. Quitter
=====
Votre choix : 3
Les datasets ont été triés par nom.

===== Datasets triés par nom =====
- maman
- papa
=====

```

Tri de la liste des datasets.

5.3 — Rechercher

```

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Trier les datasets (à venir)
4. Rechercher un dataset (à venir)
5. Modifier un dataset (à venir)
6. Supprimer un dataset (à venir)
7. Quitter
=====
Votre choix : 4
Nom du dataset à rechercher : Saori

===== Résultat de la recherche =====
Nom          : saori
Domaine      : Santé
Lignes       : 5
Colonnes     : 9
Taille       : 8.7 Mo
Format       : JSON
Public       : Non
=====

```

Recherche d'un dataset dans la liste.

5.4 — Modifier

```
Votre choix : 5
Nom du dataset à modifier : saOri

Modification du dataset 'saori'
Nouveau nom du dataset : zelda
Nouveau domaine ('Santé', 'Finance', 'Agriculture', 'Transport', 'Education') : Agriculture
Nouveau nombre de lignes : 5
Nouveau nombre de colonnes : 5
Nouvelle taille en Mo : 5
Nouveau format (csv ou json) : csv
Le dataset est-il public ? (true/false) : false

Le dataset 'saOri' a été modifié avec succès.

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Trier les datasets (à venir)
4. Rechercher un dataset (à venir)
5. Modifier un dataset (à venir)
6. Supprimer un dataset (à venir)
7. Quitter
=====
Votre choix : 2

===== Liste des datasets =====
Nom      : papa
Domaine  : Santé
Lignes   : 4
Colonnes : 5
Taille   : 9.5 Mo
Format   : CSV
Public   : Oui
-----
Nom      : zelda
Domaine  : Agriculture
Lignes   : 5
Colonnes : 5
Taille   : 5.0 Mo
Format   : CSV
Public   : Non
-----
=====
```

Modification d'un dataset existant.

5.5 — Supprimer

```

=====
Votre choix : 6
Nom du dataset à supprimer : PAPA
Le dataset 'PAPA' a été supprimé avec succès.

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Trier les datasets (à venir)
4. Rechercher un dataset (à venir)
5. Modifier un dataset (à venir)
6. Supprimer un dataset (à venir)
7. Quitter
=====
Votre choix : 2

===== Liste des datasets =====
Nom       : zelda
Domaine   : Agriculture
Lignes    : 5
Colonnes  : 5
Taille    : 5.0 Mo
Format    : CSV
Public    : Non
-----

```

Suppression d'un dataset de la liste.

Commit(s) associé(s) : ajouter une liste de datasets (partie 5.1)

trier la liste de datasets ajoutés (partie 5.2)

rechercher un dataset ajouté (partie 5.3)

modifier un dataset existant (partie 5.4)

supprimer un dataset existant (partie 5.5)

Partie 6 — Compréhensions (listes et dictionnaires)

Calcul des statistiques du catalogue à l'aide de compréhensions de listes et de dictionnaires : nombre total de datasets, nombre total de lignes, nombre moyen de colonnes, répartition public/privé, répartition par format (CSV/JSON) et répartition par domaine.

```

8. Quitter
=====
Votre choix : 7

===== Statistiques des datasets =====
Nombre de datasets      : 1
Nombre total de lignes  : 4
Nombre moyen de colonnes : 5.00
Datasets publics       : 1
Datasets privés        : 0
Datasets au format CSV  : 1
Datasets au format JSON : 0

Répartition par domaine :
- Santé : 0
- Finance : 0
- Agriculture : 0
- Transport : 0
- Education : 1
=====

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Trier les datasets
4. Rechercher un dataset
5. Modifier un dataset
6. Supprimer un dataset
7. Afficher les statistiques
8. Quitter
=====

```

Affichage des statistiques calculées sur le catalogue de datasets.

Commit(s) associé(s) : option stat dans le menu interactif + peaufinage (partie 6.)

Partie 7 — Fichiers

Sauvegarde des datasets dans un fichier datasets.csv, puis rechargement et affichage des données à partir de ce fichier.

7.1 — Sauvegarde

```

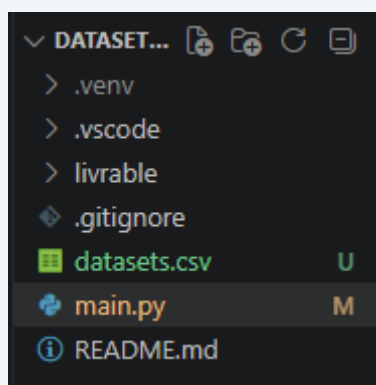
===== Statistiques des datasets =====
Nombre de datasets      : 1
Nombre total de lignes  : 5
Nombre moyen de colonnes : 4.00
Datasets publics       : 1
Datasets privés        : 0
Datasets au format CSV  : 1
Datasets au format JSON : 0

Répartition par domaine :
- Santé : 0
- Finance : 0
- Agriculture : 0
- Transport : 0
- Education : 1
=====

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Trier les datasets
4. Rechercher un dataset
5. Modifier un dataset
6. Supprimer un dataset
7. Afficher les statistiques
8. Sauvegarder les datasets
9. Recharger les datasets
10. Quitter
=====
Votre choix : 8
Les datasets ont été sauvegardés dans 'datasets.csv'.

```

Sauvegarde des datasets dans le fichier CSV.



Le fichier datasets.csv apparaît bien sur le disque après sauvegarde.

7.2 — Rechargement et affichage

```

1. Ajouter un dataset
2. Afficher les datasets
3. Trier les datasets
4. Rechercher un dataset
5. Modifier un dataset
6. Supprimer un dataset
7. Afficher les statistiques
8. Sauvegarder les datasets
9. Recharger les datasets
10. Quitter
=====
Votre choix : 9
Les datasets ont été rechargés depuis 'datasets.csv'.

===== Datasets rechargés =====
Nom       : ia
Domaine   : Education
Lignes    : 5
Colonnes  : 4
Taille    : 6.0 Mo
Format    : CSV
Public    : Oui
-----
=====

```

Rechargement et affichage des données depuis le fichier.

```

datasets.csv > data
1  nom,domaine,lignes,colonnes,taille,format,public
2  ia,Education,5,4,6.0,CSV,True
3

```

Contenu brut du fichier de données (CSV/JSON).

Commit(s) associé(s) : sauvegarder les données (partie 7.1)
 recharger et afficher les données (partie 7.2)

Partie 8 — Exceptions

Gestion des erreurs d'utilisation courantes : saisie d'un texte au lieu d'un nombre, fichier absent, dataset recherché inexistant, fichier vide — de façon à ce que le programme continue de fonctionner sans planter.

8.1 — Validation des saisies (nombres, texte)

```
def get_int(prompt):
    """
    Read a line of text from standard input and return the equivalent int;
    if text does not represent an int, user is prompted to retry. If line
    can't be read, return None.
    """
    while True:
        s = get_string(prompt)
        if s is None:
            return None
        if re.search(r"^[+-]?\d+$", s):
            try:
                return int(s, 10)
            except ValueError:
                pass
```

Fonction de saisie sécurisée pour les entiers (gestion de ValueError).

```
def get_float(prompt):
    """
    Read a line of text from standard input and return the equivalent float
    as precisely as possible; if text does not represent a double, user is
    prompted to retry. If line can't be read, return None.
    """
    while True:
        s = get_string(prompt)
        if s is None:
            return None
        if len(s) > 0 and re.search(r"^[+-]?\d*(?:\.\d*)?$", s):
            try:
                return float(s)
            except (OverflowError, ValueError):
                pass
```

Fonction de saisie sécurisée pour les nombres décimaux.

```
def get_string(prompt):
    """
    Read a line of text from standard input and return it as a string,
    sans trailing line ending. Supports CR (\r), LF (\n), and CRLF (\r\n)
    as line endings. If user inputs only a line ending, returns "", not None.
    Returns None upon error or no input whatsoever (i.e., just EOF).
    """
    if not isinstance(prompt, str):
        raise TypeError("prompt must be of type str")
    try:
        return input(prompt)
    except EOFError:
        return None
```

Fonction de saisie sécurisée pour le texte.

```
if choix == 1:
    # --- Saisie des métadonnées du dataset + Ajouter un dataset ---
    nom = get_string("Nom du dataset : ")

    domaine = get_string(f"Domaine {DOMAINES_AUTORISES} : ")
    while domaine not in DOMAINES_AUTORISES:
        print("Domaine invalide, choisissez parmi la liste proposée.\n")
        domaine = get_string(f"Domaine {DOMAINES_AUTORISES} : ")

    lignes = get_int("Nombre de lignes : ")
    colonnes = get_int("Nombre de colonnes : ")
    taille = get_float("Taille en Mo : ")
    format_fichier = get_string("Format (csv ou json) : ")
    public = get_string("Public ? (true/false) : ").strip().lower() == "true"
```

Validation de la saisie avec l'aide de la librairie CS50.

8.2 — Fichier introuvable

```
=====
1. Ajouter un dataset
2. Afficher les datasets
3. Trier les datasets
4. Rechercher un dataset
5. Modifier un dataset
6. Supprimer un dataset
7. Afficher les statistiques
8. Sauvegarder les datasets
9. Recharger les datasets
10. Quitter
=====
Votre choix : 9
Erreur : le fichier 'datasets.csv' n'existe pas.

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Trier les datasets
4. Rechercher un dataset
5. Modifier un dataset
6. Supprimer un dataset
7. Afficher les statistiques
8. Sauvegarder les datasets
9. Recharger les datasets
10. Quitter
=====
Votre choix :
```

Gestion de l'exception si le fichier datasets.csv n'existe pas.

8.3 — Dataset introuvable

```
1. Ajouter un dataset
2. Afficher les datasets
3. Trier les datasets
4. Rechercher un dataset
5. Modifier un dataset
6. Supprimer un dataset
7. Afficher les statistiques
8. Sauvegarder les datasets
9. Recharger les datasets
10. Quitter
=====
Votre choix : 4
Nom du dataset à rechercher : papa
Dataset 'papa' non trouvé.

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Trier les datasets
4. Rechercher un dataset
5. Modifier un dataset
6. Supprimer un dataset
7. Afficher les statistiques
8. Sauvegarder les datasets
9. Recharger les datasets
10. Quitter
=====
Votre choix :
```

Gestion de l'exception si le dataset recherché n'existe pas.

8.4 — Fichier vide

```

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Trier les datasets
4. Rechercher un dataset
5. Modifier un dataset
6. Supprimer un dataset
7. Afficher les statistiques
8. Sauvegarder les datasets
9. Recharger les datasets
10. Quitter
=====
Votre choix : 9
Le fichier 'datasets.csv' est vide ou ne contient aucun dataset.

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Trier les datasets
4. Rechercher un dataset
5. Modifier un dataset
6. Supprimer un dataset
7. Afficher les statistiques
8. Sauvegarder les datasets
9. Recharger les datasets
10. Quitter
=====
Votre choix :

```

Gestion de l'exception si le fichier datasets.csv est vide.

Commit(s) associé(s) : gerer les exceptions (partie 8.)

Partie 9 — Fonctions

Refactorisation du programme en fonctions dédiées : `afficher_menu()`, `ajouter_dataset()`, `afficher_datasets()`, `supprimer_dataset()`, `rechercher_dataset()`, `trier_dataset()`, `modifier_dataset()`, `sauvegarder()`, `recharger()`, `statistiques()`.

```

##### # --- Partie 2 : Menu interactif ---
# --- Partie 2 : Menu interactif (provisoire) --- fonction 0
while True:
    choix = afficher_menu()

##### # --- Partie 5.1 : Ajouter un dataset ---
# --- Partie 5.1 : Ajouter un dataset --- fonction 1
if choix == 1:
    ajouter_dataset()

##### # --- Partie 5.1 : Ajouter un dataset ---
# --- Affichage de tous les datasets --- fonction 2
elif choix == 2:
    afficher_datasets()

##### # --- Partie 5.2 : Trier les datasets ---
# --- Partie 5.2 : Trier les datasets --- fonction 5
elif choix == 3:
    trier_dataset()

##### # --- Partie 5.3 : Rechercher un dataset ---
# --- Partie 5.3 : Rechercher un dataset --- fonction 4
elif choix == 4:
    rechercher_dataset()

##### # --- Partie 5.4 : Modifier un dataset ---
# --- Partie 5.4 : Modifier un dataset --- fonction 6
elif choix == 5:
    modifier_dataset()

##### # --- Partie 5.5 : Supprimer un dataset ---
# --- Partie 5.5 : Supprimer un dataset --- Fonction 3
elif choix == 6:
    supprimer_dataset()

##### # --- Partie 6 : Statistiques ---
# --- Partie 6 : Statistiques --- fonction 9
elif choix == 7:
    statistiques()

##### # --- Partie 7.1 : Sauvegarder les données dans un fichier CSV ---
# --- Partie 7.1 : Sauvegarder les données dans un fichier CSV --- fonction 7
elif choix == 8:
    sauvegarder()

##### # --- Partie 7.2 : Recharger + afficher les données depuis un fichier CSV ---
# --- Partie 7.2 : Recharger + afficher les données depuis un fichier CSV --- fonction 8
elif choix == 9:
    recharger()

##### # --- Quitter l'application ---
elif choix == 10:
    print("Au revoir !")
    break
else:
    print("Choix invalide, veuillez réessayer.\n")

```

Programme organisé en fonctions dédiées.

Commit(s) associé(s) : créer la fonction `afficher_menu` (partie 9.1)

créer la fonction `ajouter_dataset` (partie 9.2)

créer la fonction `afficher_datasets` (partie 9.3)

créer la fonction `supprimer_dataset` (partie 9.4)

créer la fonction `rechercher_dataset` (partie 9.5)

créer la fonction `trier_dataset` (partie 9.6)

créer la fonction `modifier_dataset` (partie 9.7)

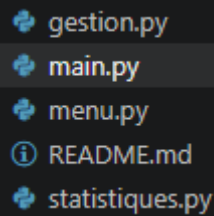
créer la fonction `sauvegarder` (partie 9.8)

créer la fonction `recharger` (partie 9.9)

créer la fonction `statistiques` (partie 9.10)

Partie 10 — Modules

Répartition du code entre plusieurs modules pour faciliter la maintenance : main.py, menu.py, gestion.py, statistiques.py.

Une image montrant une arborescence de fichiers dans un éditeur de code. Les fichiers listés sont gestion.py, main.py, menu.py, README.md et statistiques.py. main.py est sélectionné.

```
gestion.py
main.py
menu.py
README.md
statistiques.py
```

Arborescence et contenu des modules main.py / menu.py / gestion.py / statistiques.py.

Commit(s) associé(s) : faire le module menu.py (partie 10.1)

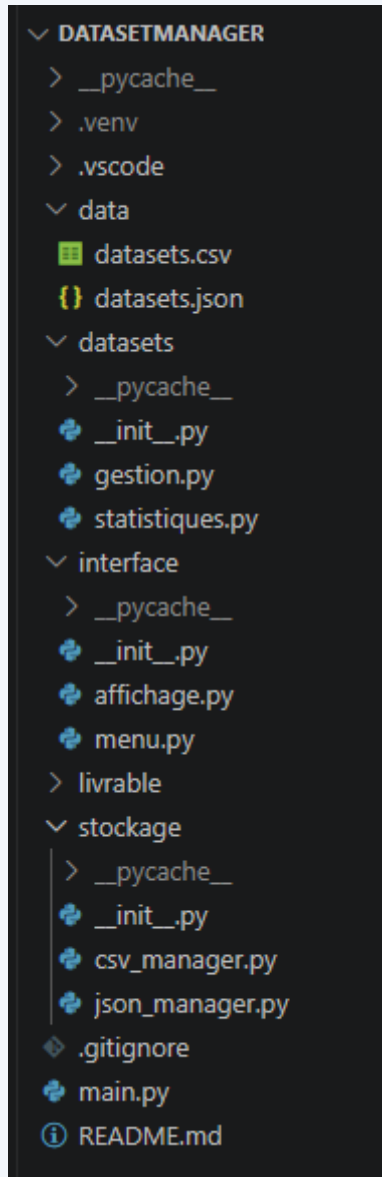
créer le module gestion.py (partie 10.2)

déplacer les fonctions dans gestion.py à l'exception de stat (partie 10.3)

adjoindre le module statistiques.py (partie 10.4)

Partie 11 — Packages

Découpage de l'application en packages : datasets/ (gestion.py, statistiques.py), interface/ (menu.py, affichage.py), stockage/ (csv_manager.py, json_manager.py), data/ (datasets.csv, datasets.json).



Arborescence finale de l'application organisée en packages.

Commit(s) associé(s) : organiser l'application en packages (partie 11.)

Partie 12 — Bonus

Fonctionnalité(s) supplémentaire(s) proposée(s) et réalisée(s) pour enrichir l'application au-delà du cahier des charges initial.

Description de la fonctionnalité bonus : Un dataset ne peut pas être ajouté si un dataset portant le même nom existe déjà.

```
#####  
# --- Partie 12 : BONUS : Vérification des doublons ---  
for dataset in datasets:  
    if dataset["nom"].lower() == nom.lower():  
        print(  
            f"Erreur : le dataset '{nom}' existe déjà. "  
            "Veuillez choisir un autre nom.\n"  
        )  
    return  
#####
```

Démonstration de la fonctionnalité bonus.

Commit(s) associé(s) : `empêcher les doublons 'nom' de datasets BONUS (partie 12.)`

Conclusion

Ce projet m'a permis de mettre en pratique les principaux concepts du langage Python, notamment les structures de données, les fonctions, la gestion des fichiers, les exceptions ainsi que l'organisation d'un projet en modules et packages en + d'une familiarité construite avec Git et GitHub grâce à un développement progressif accompagné de commits réguliers.